

International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory**Lab 4 – C++ Programming: Friend Function and Friend Class****Date:** 21.08.2026**Semester:** B.Tech 3rd Semester**Section:** CSE B1**Instructions**

- Write all programs in C++.
- Use the **friend function** or **friend class** concept as specified in each question.
- Keep the required data members **private**.
- Use appropriate classes, objects, member functions, and access specifiers.
- Display the output in a neat and readable format.
- Ensure that the friend function or friend class is used meaningfully to access private members.

1. Weather Report – Friend Function

Create a class named Weather containing the following private data members:

- City Name
- Temperature
- Weather Condition

Write a friend function named `generateReport()` that accesses the private members and displays a suitable weather report.

The function should classify the temperature as follows:

- Above 35°C – **Very Hot**
- 20°C to 35°C – **Pleasant**
- Below 20°C – **Cool**

***Hint:** The friend function should directly access the private temperature member and perform the classification.*

2. Two-Factor Login – Friend Function

Create a class named UserAccount containing the following private data members:

- Username
- Login Attempts
- Account Status

Write a friend function named `checkAccount()` that accesses the private members and checks the account status.

If the number of unsuccessful login attempts is 3 or more, display **“Account Locked”**; otherwise display **“Account Active”**.

The function should also display the username and number of login attempts.

Hint: Keep all data members private and allow the friend function to access them directly.

3. Compare Two Digital Cameras – Friend Function

Create a class named `Camera` containing the following private data members:

- Brand
- Model
- Megapixels
- Storage Capacity

Create two `Camera` objects. Write a friend function named `compareCamera()` that determines which camera is better based on the following conditions:

1. The camera with higher megapixels is considered better.
2. If both cameras have the same megapixels, the camera with higher storage capacity is considered better.

Display the details of the better camera.

Hint: Pass both `Camera` objects to the friend function and compare their private members.

4. Electricity Usage Alert – Friend Function

Create a class named `ElectricMeter` containing the following private data members:

- Meter Number
- Consumer Name
- Units Consumed

Write a friend function named `checkUsage()` that accesses the private members and categorizes electricity usage as follows:

- Below 100 units – **Low Usage**
- 100 to 300 units – **Moderate Usage**
- Above 300 units – **High Usage**

Display the consumer details and the corresponding usage category.

Hint: Use conditional statements inside the friend function to classify the consumption.

5. Event Registration Verification – Friend Function

Create a class named `EventParticipant` containing the following private data members:

- Participant Name
- Age
- Registration Status

Write a friend function named `verifyParticipant()` that determines whether the participant is eligible for the event.

A participant is eligible only if:

- The participant is 18 years or older.
- The registration status is active.

Display the participant details and either **“Eligible”** or **“Not Eligible”**.

Hint: The friend function should access both the age and registration status directly.

6. Printer Control System – Friend Class

Create two classes named `Printer` and `PrinterManager`.

The `Printer` class should contain the following private data members:

- Printer Name
- Number of Pages Printed
- Ink Level
- Power Status

Declare `PrinterManager` as a **friend class** of `Printer`.

The `PrinterManager` class should provide member functions to:

1. Display printer information.
2. Turn the printer ON.
3. Turn the printer OFF.
4. Check the ink level.
5. Reset the page count.

Hint: Since `PrinterManager` is a friend class, its member functions can directly access and modify the private members of `Printer`.

7. Museum Exhibit Controller – Friend Class

Create two classes named `Exhibit` and `MuseumManager`.

The `Exhibit` class should contain the following private data members:

- Exhibit Name
- Exhibit ID
- Visitor Count
- Display Status

Declare `MuseumManager` as a **friend class** of `Exhibit`.

The `MuseumManager` class should provide member functions to:

1. Display exhibit information.

2. Add visitors to the exhibit.
3. Reset the visitor count.
4. Open or close the exhibit.
5. Display whether the exhibit is currently open.

Hint: The friend class should directly update the visitor count and display status.

8. Vehicle Service Tracker – Friend Class

Create two classes named `VehicleService` and `ServiceManager`.

The `VehicleService` class should contain the following private data members:

- Vehicle Number
- Owner Name
- Service Due Status
- Last Service Kilometres

Declare `ServiceManager` as a **friend class** of `VehicleService`.

The `ServiceManager` class should provide member functions to:

1. Display vehicle service information.
2. Mark the service as completed.
3. Update the last service kilometres.
4. Check whether the vehicle requires servicing.

Hint: Use the private service status and last service kilometres to determine whether servicing is required.

9. Digital Wallet Controller – Friend Class

Create two classes named `DigitalWallet` and `WalletManager`.

The `DigitalWallet` class should contain the following private data members:

- User Name
- Wallet Balance
- Wallet Status

Declare `WalletManager` as a **friend class** of `DigitalWallet`.

The `WalletManager` class should provide member functions to:

1. Display wallet details.
2. Add money to the wallet.
3. Deduct money from the wallet if sufficient balance exists.
4. Disable the wallet.
5. Display the current wallet status.

***Hint:** Before deducting money, check whether the wallet has sufficient balance. The friend class can directly modify the private balance and status.*

10. Classroom Attendance Manager – Friend Class

Create two classes named `Classroom` and `AttendanceManager`.

The `Classroom` class should contain the following private data members:

- Class Name
- Total Students
- Present Students
- Attendance Status

Declare `AttendanceManager` as a **friend class** of `Classroom`.

The `AttendanceManager` class should provide member functions to:

1. Display classroom information.
2. Update the number of present students.
3. Mark the class attendance as completed.
4. Display whether attendance has been completed.
5. Calculate and display the number of absent students.

***Hint:** Calculate the number of absent students using:*

$$\text{Absent Students} = \text{Total Students} - \text{Present Students}$$